

3주차 과제2

사람

왕은서(엘텍공과대학 소프트웨어학부)

오늘 한 일 요약

- 3주차 과제(정적 분석 모듈 구현) 중 내 담당인 B — 디컴파일 연동 구현.

담당 역할: B — 디컴파일 연동

| `subprocess.run()` 으로 `apktool d`, `jadx` CLI 호출하는 wrapper 함수 작성 — 타임아웃 설정, 디컴파일 실패 시 예외 처리

구현 내용

파일 위치: `src/static_analyzer/decompiler.py`, `src/static_analyzer/__init__.py`

제공 함수/예외:

- `run_apktool(apk_path, output_dir, timeout=120)` → Path — `apktool d -f -o <output_dir> <apk_path>` 실행
- `run_jadx(apk_path, output_dir, timeout=120)` → Path — `jadx -d <output_dir> <apk_path>` 실행
- `DecompileError` — 디컴파일 실패(도구 없음, 잘못된 종료 코드 등) 시 발생하는 기본 예외
- `DecompileTimeoutError(DecompileError)` — 지정한 시간 안에 안 끝났을 때 발생

설계 이유

- 예외를 2개로 분리한 이유: 타임아웃과 일반 실패를 구분해두면, 나중에 A가 만들 오케스트레이터(`analyze_static`)에서 "타임아웃이면 재시도, 그 외 실패면 즉시 중단" 같은 다른 처리를 할 수 있음.
- `apktool -f 옵션`: 같은 출력 폴더로 재실행할 때 "이미 존재함" 에러가 나지 않도록 강제 덮어쓰기.
- `capture_output=True, text=True`: 실패 시 `stderr` 내용을 예외 메시지에 그대로 포함시켜서, 나중에 로그만 보고도 왜 실패했는지 바로 알 수 있게 함.
- APK 파일 존재 여부를 `subprocess` 호출 전에 먼저 체크: 존재하지 않는 파일을 CLI에 넘기면 도구마다 에러 메시지 형식이 달라서 파싱이 번거로움. Python 쪽에서 먼저 걸러서 명확한 `FileNotFoundError` 로 통일.

코드

```
"""apktool / jadx CLI를 감싸는 디컴파일 wrapper 함수 모음."""

from __future__ import annotations

import subprocess
from pathlib import Path

DEFAULT_TIMEOUT = 120 # 초. 큰 APK는 더 걸릴 수 있으므로 호출부에서 조정 가능

class DecompileError(Exception):
    """디컴파일 도구 실행이 실패했을 때 발생."""

class DecompileTimeoutError(DecompileError):
    """지정한 시간 안에 디컴파일이 끝나지 않았을 때 발생."""

def _run_tool(cmd: list[str], tool_name: str, timeout: int) -> subprocess.CompletedPro
```

```

cess:
    try:
        return subprocess.run(cmd, capture_output=True, text=True, timeout=timeout)
    except subprocess.TimeoutExpired as e:
        raise DecompileTimeoutError(
            f"{tool_name}이(가) {timeout}초 내에 끝나지 않아 중단함: {' '.join(cmd)}"
        ) from e
    except FileNotFoundError as e:
        raise DecompileError(
            f"'{cmd[0]}' 실행 파일을 찾을 수 없음. PATH에 {tool_name} 경로가 등록되어 있는지 확인하세요."
        ) from e

def run_apktool(apk_path: str | Path, output_dir: str | Path, timeout: int = DEFAULT_TIMEOUT) -> Path:
    apk_path = Path(apk_path)
    output_dir = Path(output_dir)
    if not apk_path.is_file():
        raise FileNotFoundError(f"APK 파일을 찾을 수 없음: {apk_path}")

    cmd = ["apktool", "d", "-f", "-o", str(output_dir), str(apk_path)]
    result = _run_tool(cmd, "apktool", timeout)

    if result.returncode != 0:
        raise DecompileError(
            f"apktool 디컴파일 실패 (exit code {result.returncode}): {result.stderr.strip()}"
        )
    return output_dir

def run_jadx(apk_path: str | Path, output_dir: str | Path, timeout: int = DEFAULT_TIMEOUT) -> Path:
    apk_path = Path(apk_path)
    output_dir = Path(output_dir)
    if not apk_path.is_file():
        raise FileNotFoundError(f"APK 파일을 찾을 수 없음: {apk_path}")

    cmd = ["jadx", "-d", str(output_dir), str(apk_path)]
    result = _run_tool(cmd, "jadx", timeout)

    if result.returncode != 0:
        raise DecompileError(
            f"jadx 디컴파일 실패 (exit code {result.returncode}): {result.stderr.strip()}"
        )
    return output_dir

```

검증 방법

지금 작업한 리눅스 환경엔 apktool/jadx가 실제로 설치되어 있지 않아서(우리 팀은 다들 Windows/에뮬레이터 기준으로 작업 중), 진짜 APK로는 아직 테스트 못함. 대신 가짜 실행 파일(bash 스크립트)로 apktool/jadx 동작을 흉내내서 **4가지 실패 케이스**가 의도한 예외로 정확히 이어지는지 확인함:

케이스	트리거	결과
APK 경로가 존재하지 않음	없는 파일 경로 전달	<code>FileNotFoundError</code> ✓
apktool/jadx가 PATH에 없음	PATH에서 바이너리 제거	<code>DecompileError</code> (실행 파일 못 찾을 메시지) ✓
타임아웃 초과	5초 걸리는 가짜 도구 + <code>timeout=1</code>	<code>DecompileTimeoutError</code> ✓
도구가 비정상 종료	<code>exit 1</code> • stderr 출력하는 가짜 도구	<code>DecompileError</code> (stderr 메시지 포함) ✓

네 가지 모두 의도대로 동작 확인함.

⚠ **남은 검증:** 실제 apktool/jadx가 깔린 내 Windows PC에서 `anubis.apk`, `syssecapp.apk` 같은 진짜 샘플로 한 번 더 실행해서 정상 디컴파일 케이스(성공 경로)도 확인 필요. 지금까지는 실패 경로만 검증됨.

다음에 할 일 (B 완료 후)

- A의 인터페이스 확정 기다리기 / 확인하기** — `analyze_static(apk_path) -> dict` 형식이 아직 안 정해졌으면, 이게 먼저 나와야 내 wrapper 함수를 오케스트레이터에 연결할 수 있음. A한테 상태 물어보기.
- C에게 output_dir 구조 공유** — C는 `AndroidManifest.xml` 을 파싱해야 하는데, `run_apktool` 결과 폴더(`output_dir`) 밑에 `AndroidManifest.xml` 이 그대로 생김. 이 경로 형식을 C와 미리 맞춰야 함 (androguard를 쓰면 사실 apktool 없이 apk 파일 자체에서 바로 파싱 가능하니, C가 apktool 결과물이 필요한지부터 확인하면 좋음).
- 실제 apk로 성공 경로 검증** — 위 남은 검증 항목.
- 팀 4명 산출물 취합** — A(인터페이스), B(디컴파일 wrapper, 완료), C(Manifest 파싱), D(점수화/출력) 각자 작성한 함수를 `static_analyzer/` 패키지 하나로 합치기. 지금은 팀 git이 아직 연결 안 된 상태(CLAUDE.md 기준)라, 합치는 방식(공유 폴더로 파일 주고받기 vs git 연결부터 하기)을 먼저 정하는 게 좋음.
- 1차 발표 데모 준비** — 내 파트는 "apktool/jadx 실행 → 산출물 폴더 생성"을 터미널 캡처나 라이브 시연으로 보여줄 수 있음. 위 3번(실제 apk 테스트)이 끝나면 바로 준비 가능.

관련 자료

- [이삼이오 방캅스 3주차](#) — 팀 과제 원문 및 역할 분배

구현한 것

src/static_analyzer/decompiler.py에 요구사항대로 작성했습니다.

- `run_apktool(apk_path, output_dir, timeout=120)`, `run_jadx(apk_path, output_dir, timeout=120)` — `subprocess.run()`으로 각 CLI 호출
- `DecompileError` / `DecompileTimeoutError` 두 개의 예외로 "타임아웃"과 "그 외 실패(도구 없음, 비정상 종료)"를 구분 — 나중에 A의 오케스트레이터가 타임아웃일 땐 재시도, 나머지는 즉시 중단 식으로 다르게 처리할 수 있도록

이 리눅스 환경엔 apktool/jadx가 실제로 안 깔려있어서, 가짜 실행 파일로 4가지 실패 케이스(APK 없음, 도구 없음, 타임아웃, 비정상 종료)를 만들어 예외 처리가 의도대로 동작하는지 확인했습니다 — 전부 통과. 다만 진짜 apk로는 아직 검증 못 했으니, apktool/jadx가 깔린 본인 Windows PC에서 `anubis.apk` 등으로 한 번 더 돌려봐야 합니다.

B 끝난 다음에 할 일

- A의 인터페이스 확정 확인** — `analyze_static(apk_path) -> dict` 형식이 아직 안 나왔으면, 이게 나와야 내 wrapper 함수를 오케스트레이터에 연결할 수 있음. A한테 진행 상황 물어보기.
- C한테 output_dir 구조 공유** — apktool 결과 폴더 안에 `AndroidManifest.xml`이 그대로 생기는데, C가 이 파일을 파싱할 건지 아니면 apk 파일 자체에서 바로 파싱할 건지(androguard는 apktool 없이도 가능) 미리 맞춰야 함.
- 진짜 apk로 wrapper 테스트** — 위 검증의 남은 부분.
- 팀원 4명 산출물 취합** — CLAUDE.md에 팀 git이 아직 연결 안 됐다고 되어 있어서, 지금은 파일 공유든 git 연결이든 합치는 방법부터 팀이 정해야 함.
- 1차 발표 데모 준비** — 내 파트는 "apktool/jadx 실행 → 산출물 폴더 생성"을 터미널로 보여주는 정도로 준비 가능.